

NumPy

Numerical Python — Complete Notes

Compiled & Curated by

Muhammad Saad

AI & Data Science Learner • Bano Qabil • PNY Trainings

#NumPy

#Python

#DataScience

#AI

#NeverStopLearning

"I don't wait for the right time to learn — I make every time the right time."

1. What is NumPy?

NumPy (Numerical Python) is an open-source Python library used for **numerical computing**. It provides tools for working efficiently with arrays, matrices, and mathematical functions.

NumPy introduces the **ndarray** (n-dimensional array) — much faster and more memory-efficient than Python lists.

Important Features

- **Multidimensional Arrays:** Efficient storage and manipulation of large datasets — 1D, 2D, 3D arrays.
- **Mathematical Functions:** Linear algebra, Fourier transforms, random number generation, and more.
- **Vectorization:** Operations on entire arrays without explicit Python loops — cleaner and faster.
- **Interoperability:** Works seamlessly with Pandas, Matplotlib, TensorFlow, PyTorch, etc.

Basic Usage

```
import numpy as np

arr = np.array([1, 2, 3, 4, 5])
print(arr) # [1 2 3 4 5]
```

2. NumPy Arrays vs Python Lists

Feature	Python List	NumPy Array
Speed	Slow ■	Up to 50x Faster ■
Memory	More memory	Contiguous — less memory
Data Types	Mixed types allowed	Homogeneous (same type)
Operations	Manual loops needed	Vectorized — no loops
C Implementation	Pure Python	Built on C — blazing fast

- *NumPy arrays are implemented in C internally — that's why they're so fast!*

3. Creating NumPy Arrays

From Python Lists

```
arr = np.array([1, 2, 3, 4]) # 1D array
arr2D = np.array([[1,2,3],[4,5,6]]) # 2D array (matrix)
arr3D = np.array([[[1,2],[3,4]],[[5,6],[7,8]]]) # 3D array
```

Using Built-in Functions

np.zeros((3,4)) → 3x4 array filled with 0s

`np.ones((3,3))` → 3x3 array filled with 1s
`np.full((2,3), 5)` → 2x3 array filled with 5s
`np.eye(3)` → 3x3 Identity matrix
`np.arange(1, 20, 2)` → Array: elements from 1 to 20 step 2
`np.linspace(0, 10, 5)` → 5 evenly spaced values between 0 and 10

4. Array Properties

```
arr = np.array([[1,2,3],[4,5,6],[7,8,9],[10,11,12]])

arr.shape # (4, 3) – rows x cols
arr.size # 12 – total elements
arr.ndim # 2 – number of dimensions
arr.dtype # int64 – data type
arr.itemsize # 8 – bytes per element
```

5. Operations on Arrays

Reshaping

```
arr = np.array([1,2,3,4,5,6])
reshaped = arr.reshape((2,3)) # 1x6 → 2x3
flattened = reshaped.flatten() # 2D → 1D
```

Indexing

```
arr = np.array([1,2,3,4,5])
arr[0] # 1 (first element)

arr2D = np.array([[1,2,3],[4,5,6]])
arr2D[0][1] # 2
arr2D[1][2] # 6
```

Fancy & Boolean Indexing

```
idx = [0, 1, 4]
arr[idx] # Fancy indexing – picks elements at positions 0,1,4

arr[arr > 2] # Boolean – all elements greater than 2
arr[arr % 2 == 0] # Boolean – even numbers only
```

Slicing

```
arr = np.array([1,2,3,4,5,6,7])
arr[2:6] # [3,4,5,6]
arr[:6] # [1,2,3,4,5,6]
arr[3:] # [4,5,6,7]
arr[::2] # [1,3,5,7] – every 2nd element
```

■ *Slicing a NumPy array returns a VIEW (not a copy) — modifying it changes the original! Use `.copy()` to avoid this.*

6. Multi-dimensional Arrays

NumPy arrays can have **any number of dimensions**. Each dimension is called an **axis**.

1D Array: 1 axis (axis0)

2D Array (Matrix): 2 axes — axis0=rows, axis1=columns

3D Array (Tensor): 3 axes — axis0=depth, axis1=rows, axis2=columns

■ *In ML/DL, images are stored as 3D arrays: grayscale = height×width (2D), color (RGB) = height×width×channels (3D)*

7. Data Types in NumPy (dtype)

int32, int64: Integer values

float32, float64: Decimal values — ML libraries usually expect float32

bool: True / False

complex64, complex128: Complex numbers

S (byte-str), U (unicode-str): String types

object: Generic Python objects

```
arr = np.array([1,2,3], dtype='float64')
new_arr = arr.astype(np.int64) # change type
```

■ *Use Downcasting to save memory — e.g. ages (0-120) as int8 instead of int64 saves 8x memory!*

8. Vectorization & Broadcasting

Vectorization

Vectorization means performing operations on entire arrays at once — without explicit Python loops. Uses C-level implementations internally.

```
arr = np.array([1,2,3,4,5])
sq_arr = arr ** 2 # [1, 4, 9, 16, 25] – no loop needed!
arr2 = np.array([6,7,8,9,10])
print(arr + arr2) # [7, 9, 11, 13, 15]
```

Broadcasting

Broadcasting allows NumPy to automatically expand arrays of different shapes for arithmetic operations — without extra memory.

```
arr_mul10 = arr * 10 # Multiply every element by 10 – scalar broadcast

arr1D = np.array([1,2,3])
arr2D = np.array([[1,2,3],[4,5,6]])
print(arr1D + arr2D) # Broadcasts 1D across each row of 2D
```

■ *Broadcasting Rule: dimensions must be equal, 1, or missing (can be 'stretched').*

Standard Vector Normalization (used in ML!)

```
mean = np.mean(arr)
std_dev = np.std(arr)
normalized = (arr - mean) / std_dev # mean=0, std=1
```

9. Mathematical Functions

Aggregation Functions

np.sum() → Sum of all elements

np.prod() → Product of all elements

np.min() / **np.max()** → Min / Max value

np.argmin() / **np.argmax()** → Index of min / max

np.mean() → Average (mean)

np.median() → Median value

np.std() → Standard deviation

np.var() → Variance

Power, Log & Rounding Functions

np.square(arr) → Square of all elements

np.sqrt(arr) → Square root

np.pow(arr, 3) → Power — arr^3

np.log() / **np.log10()** / **np.log2()** → Natural / Base-10 / Base-2 log

np.exp(arr) → e^x for each element

np.round() / **np.floor()** / **np.ceil()** → Round / Floor / Ceil

np.abs(arr) → Absolute value

np.sort(arr) → Sort in increasing order

np.unique(arr) → Return unique elements

Keep Learning.
Keep Building.
Keep Sharing.

Notes compiled by **Muhammad Saad**

AI & Data Science Journey • Bano Qabil • PNY Trainings

"Insha Allah — very soon you will hear: Saad is an AI & Data Science Engineer."